HOSPITAL MANAGEMENT SYSTEM

Technical Architecture & Implementation Document

Vanguard HMS — Full-Stack Platform

Project: Hospital Management System — Backend, Angular Web & Mobile

Document Type: Technical Architecture & Implementation Record

Version: 1.0

Date: July 2026

TABLE OF CONTENTS

1. PROJECT OVERVIEW

1.1 PURPOSE

The Hospital Management System (HMS) is a full-stack platform that automates the core operations of a hospital. It is delivered as three integrated applications over a single authoritative backend: a Node.js and Express REST API, an Angular web application for staff, and a React Native app for patients. The backend is the single source of truth for all data and business rules, and both clients communicate with it exclusively over authenticated HTTPS. This document records the architecture as implemented—how each application is structured, how they interact, and how authentication, authorization, data modelling, and deployment are handled.

1.2 OBJECTIVES

- Provide role-based CRUD operations across every clinical and administrative module.
- Enforce approval workflows for staff self-registration and self-service profile changes.
- Secure the platform with JSON Web Tokens, rotating refresh tokens, a fine-grained permission model, and a complete audit trail.
- Offer patients self-service booking, record viewing, and profile management on mobile.

1.3 BUSINESS PROBLEM

Hospitals coordinate many staff, patients, appointments, and records under strict access-control and accountability requirements. HMS addresses this with a centralized API that encodes the authorization policy once and applies it uniformly to every request, keeping views consistent and every privileged action accountable.

1.4 SYSTEM OVERVIEW AND MAJOR COMPONENTS

The platform follows a three-tier design: staff use the Angular admin panel and patients the Expo mobile app, both reaching the Express API, which persists state in MongoDB through Mongoose and sends email through Brevo. Figure 2.1 and Table 1.1 summarize the arrangement.

Table 1.1: Major Platform Components

Component	Technology	Primary Responsibility
Backend API	Node.js · Express 5	Authentication, RBAC, business logic, validation, auditing, email (Brevo)
Admin Web App	Angular 21	Staff interface: employees, patients, appointments, records, approvals
Mobile App	React Native · Expo SDK 56	Patient self-service: booking, records, profile
Database	MongoDB · Mongoose	Persistent store for all collections

2. OVERALL SYSTEM ARCHITECTURE

2.1 HIGH-LEVEL ARCHITECTURE

The system is a hub-and-spoke arrangement with the Express API at the hub. The clients never communicate with each other or the database directly; every operation is mediated by the API, which applies the same authentication, authorization, validation, and auditing pipeline to all callers, so the access-control policy cannot be bypassed by a misbehaving client.

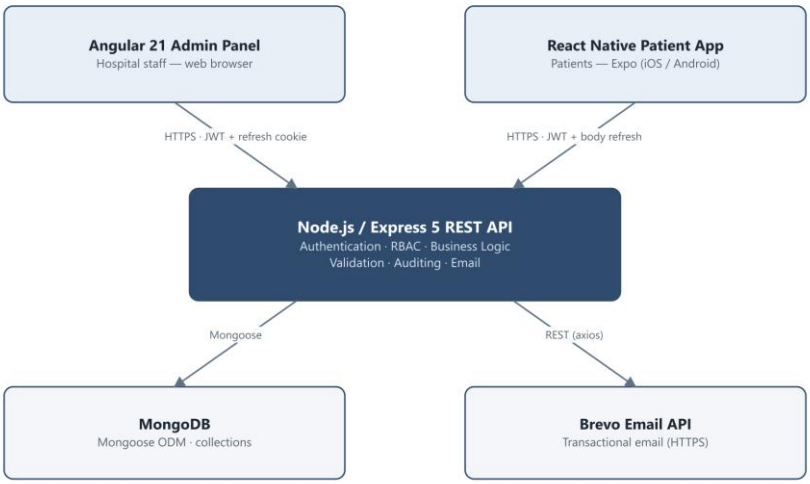

Figure 2.1: Overall System Architecture

2.2 COMPONENT INTERACTION

The two clients differ mainly in session storage—the Angular app keeps its access token in memory with a first-party HttpOnly refresh cookie, while the mobile app uses encrypted secure storage—but both attach the access token as a Bearer credential and receive the same validation, authorization, execution, auditing, and JSON-envelope treatment from the API.

2.3 OVERALL DATA FLOW

A representative write—a receptionist booking an appointment—illustrates the path: after authentication, the node and permission middleware confirm the caller may reach the appointments module and holds CREATE_APPOINTMENT; the controller validates the slot; Mongoose writes the document; and an audit record and confirmation email are produced before the resource is returned. Because all clients share one backend, an action on one is immediately visible to the others (Section 11).

3. BACKEND ARCHITECTURE

The backend is a Node.js application built on Express 5. It is organized as a layered, model–view–controller-style service in which routing, middleware, controllers, models, and cross-cutting utilities each have a clearly bounded responsibility. Express 5 automatically forwards rejected promises from asynchronous route handlers to the error middleware, so controllers throw typed errors directly instead of wrapping every handler.

3.1 TECHNOLOGY STACK

Table 3.1: Backend Technology Stack

Technology	Purpose
Node.js / Express 5	Runtime and web framework (async errors auto-forwarded)
MongoDB / Mongoose 9	Document database and object-document mapper
jsonwebtoken	Signs and verifies HS256 access tokens
bcryptjs	One-way password hashing (10–12 salt rounds)
express-validator	Declarative request validation
helmet	Secure HTTP response headers
cors	Cross-origin policy with credentials
express-rate-limit	Throttling of login and password-reset routes
cookie-parser	Reads the HttpOnly refresh cookie
morgan	HTTP request logging
axios	Calls the Brevo email REST API

3.2 FOLDER STRUCTURE

The source tree under src/ separates each concern into its own directory: controllers hold request-handling logic; models define the fourteen Mongoose schemas; routes wire endpoints to middleware and controllers; middlewares implement cross-cutting policy (authentication, node and permission authorization, validation, error handling, and rate limiting); validators declare input rules; utils provide reusable building blocks such as the token service, audit helper, pagination, and the soft-delete plugin; constants centralize the permissions catalog, enumerations, and messages; and config bootstraps environment validation and the database

connection. A thin serverless entry under src/api reuses the same Express application for the managed deployment.

3.3 LAYERED AND MVC ARCHITECTURE

Requests descend through well-defined layers and responses ascend back through them. The entry layer registers global middleware; the routing layer selects an endpoint; the middleware layer authenticates and authorizes; the controller layer executes business logic; and the model layer reads and writes MongoDB. Utilities, validators, constants, and the error handler are cross-cutting and are used by every layer. Figure 3.1 depicts this arrangement.

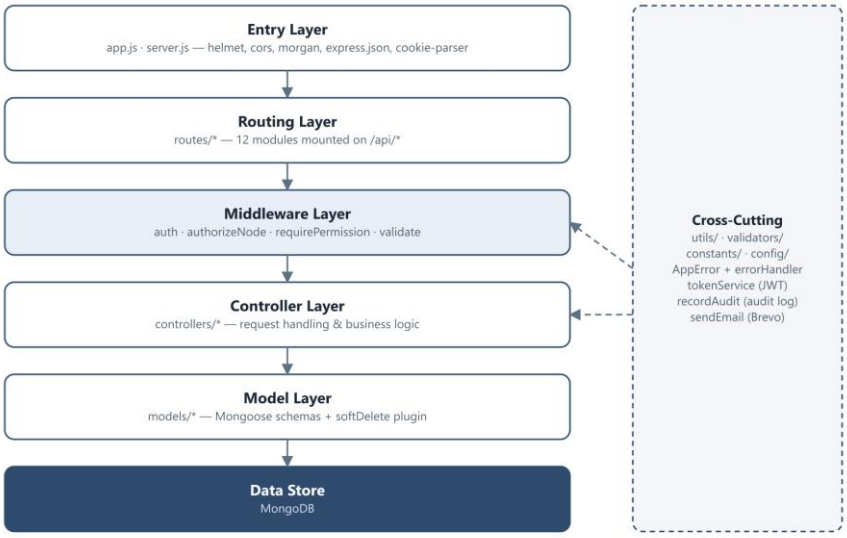

Figure 3.1: Backend Layered Architecture

3.4 REQUEST LIFECYCLE

Every protected request passes through the same ordered pipeline before any business logic runs, rate limiting protects sensitive authentication routes; validation rejects malformed input with a 422; authentication verifies the JWT and re-checks the account in the database; the node door confirms the caller may reach the module; and the permission check confirms the specific action is allowed. Any error thrown at any stage is caught by the global error handler and rendered as a consistent JSON envelope. Figure 3.2 shows the full lifecycle.

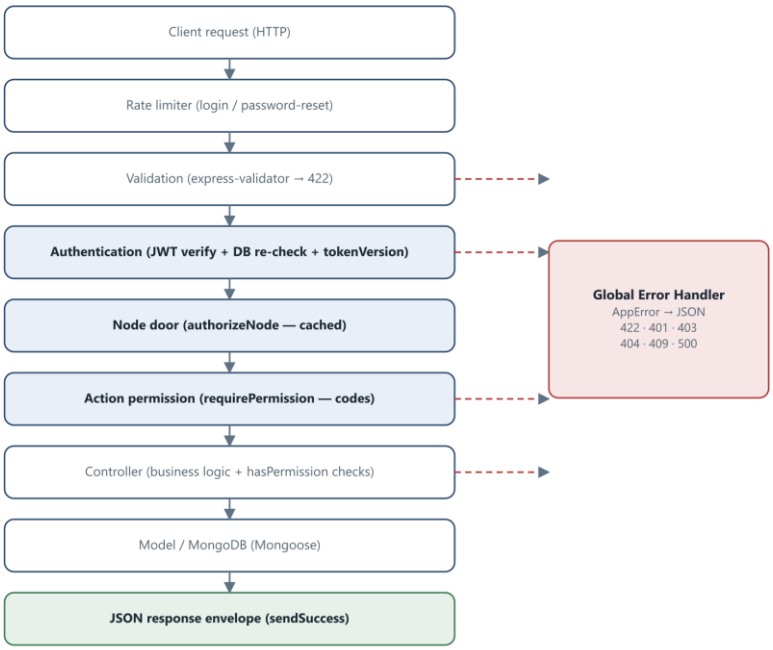

Figure 3.2: Backend Request Lifecycle

3.5 CONTROLLERS, ROUTES, AND MODELS

Routes are grouped by business domain and mounted under a common /api prefix, each attaching its validator set and the middleware chain appropriate to its sensitivity before delegating to a controller. Controllers orchestrate validation results, enforce record-level scoping (for example, restricting doctors to their own appointments and records), invoke the models, write audit entries, and return responses through a shared success helper. Models centralize field definitions, enumerations, indexes, and pre-save hooks; all human-readable identifiers such as EMP-000001 or APT-000042 are generated atomically from a shared Counter collection.

3.6 MIDDLEWARE AND VALIDATION

Middleware encapsulates the policies that apply uniformly across many routes. The authentication middleware verifies the Bearer token, confirms account status and token version, and attaches the decoded identity to the request; two authorization middlewares enforce the two-layer access model described in Section 8; the validation middleware runs the express-validator chain and raises a 422 with field-level messages; the rate limiters throttle the login (ten per fifteen minutes) and password-reset (five per fifteen minutes) routes; and coarse role and designation guards protect the dashboard and owner routes.

3.7 UTILITIES AND ERROR HANDLING

Cross-cutting utilities keep controllers small and consistent: a typed AppError class carries an HTTP status, machine code, and optional field errors; a token service signs access tokens and issues, rotates, and revokes opaque refresh tokens; an audit helper records privileged actions without ever throwing; and a soft-delete plugin hides logically deleted documents from ordinary queries. The global error handler recognizes AppError instances, Mongoose validation and cast errors, duplicate-key violations, and malformed JSON, mapping each to the correct status code.

4. DATABASE ARCHITECTURE

HMS stores all state in MongoDB, accessed exclusively through Mongoose schemas. The data model is normalized around stable business identifiers rather than raw ObjectIds: users, employees, patients, appointments, and records reference one another through human-readable codes, which keeps documents self-describing and simplifies auditing.

4.1 COLLECTIONS

Table 4.1: Core MongoDB Collections

Collection	Purpose	Key Fields
User	Login credentials and account state	employeeCode, email, roles[], status, tokenVersion, mustChangePassword
Employee	Staff and doctor profiles	employeeCode, name, designation, department, availabilitySlots
Patient	Patient profiles and credentials	UHID, name, gender, dob, address, emergencyContact, status
Appointment	Doctor–patient bookings	appointmentId, patientUHID, doctorEmployeeId, date, timeSlot, status
MedicalRecord	Diagnosis and treatment	medicalRecordId, appointmentId, prescriptionItems, status
Permission	Action permissions per designation	designation, permissions[]
Node	Sidebar / module access by designation	nodeId, name, path, allowedDesignations[]
ProfileChangeRequest	Pending self-service edits	requestId, employeeCode, requestedChanges, status
AuditLog	Immutable record of privileged actions	auditId, actorType, action, targetId, message
RefreshToken	Rotating session tokens	tokenHash, subjectType, familyId, expiresAt (TTL)
Counter	Atomic sequence source for all IDs	name, seq

4.2 ENTITY RELATIONSHIPS

A user record links one-to-one with an employee through the shared employeeCode, separating credentials from profile data. Each appointment references its patient by UHID and its doctor by employeeCode, and each medical record references its originating appointment one-to-one

plus the patient and treating doctor. Refresh tokens reference their subject (an employee or patient) and are grouped into families for reuse detection. Figure 4.1 presents the full entity-relationship diagram.

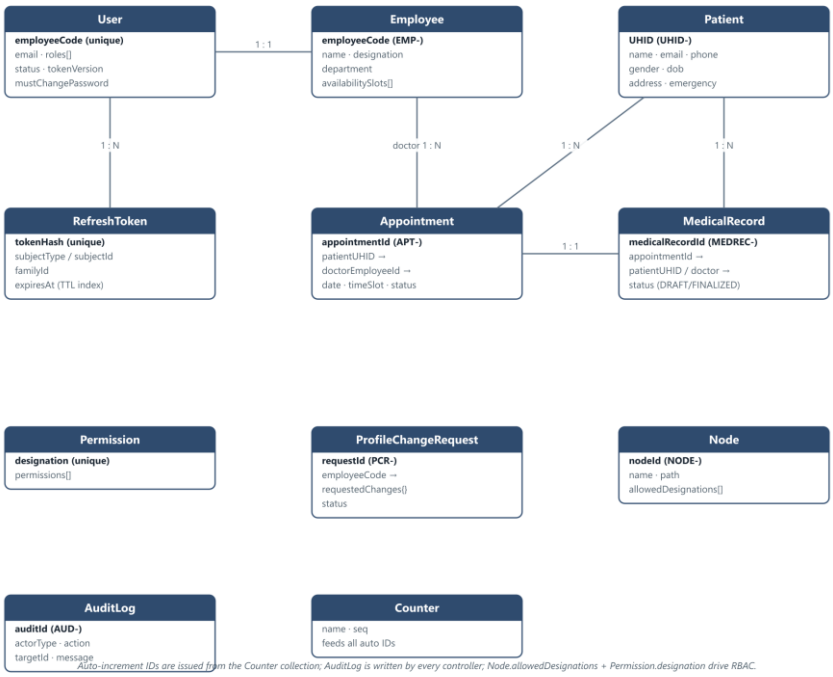

Figure 4.1: Database Entity-Relationship Diagram

4.3 IDENTIFIERS, INDEXES, AND SOFT DELETION

Every collection that exposes a business identifier obtains it from the Counter collection inside a pre-save hook, which atomically increments a named sequence and zero-pads it, yielding stable codes such as UHID-000007 or MEDREC-000031. Unique indexes protect these codes and the employeeCode key, and a TTL index on the refresh-token expiry automatically purges expired sessions. Deletion is logical rather than physical: a shared soft-delete plugin adds isDeleted, deletedAt, and deletedBy fields and injects an isDeleted filter into ordinary queries, so removed employees, patients, and users disappear from listings while remaining available for audit. Medical records implement the same pattern inline. This preserves referential history and supports the audit requirements of a clinical system.

5. ANGULAR WEB APPLICATION ARCHITECTURE

The staff-facing web client is an Angular 21 application built entirely from standalone components with zoneless change detection. State is expressed with Angular signals rather than an external store, and the UI is intentionally dependency-light: toasts, confirmation dialogs, pagination, and searchable selects are all hand-built components rather than third-party libraries.

5.1 TECHNOLOGY STACK

Table 5.1: Angular Technology Stack

Technology	Purpose
Angular 21 (standalone, zoneless)	Core framework with signal-based reactivity
Angular Router	Lazy-loaded, guard-protected navigation
Angular HttpClient + functional interceptor	API communication and token handling
Angular Reactive Forms	Typed forms with custom validators
Angular Signals	Component and shared state management
RxJS	HTTP streams and refresh-token coordination
Vitest	Unit testing

5.2 APPLICATION STRUCTURE

The application is organized into three top-level areas. The core directory contains singletons: route guards, the HTTP interceptor, API services, domain models, and shared validators. The features directory contains the routed pages, grouped into an authentication group and a dashboard group that spans employees, admins, approvals, patients, appointments, medical records, permissions, and menu nodes. The shared directory contains reusable presentational UI—navbar, sidebar, dashboard layout, toast, confirm modal, pagination, searchable select, slot picker, availability calendar, and the medical-record dialogs. Figure 5.1 summarizes this structure and the data path to the backend.

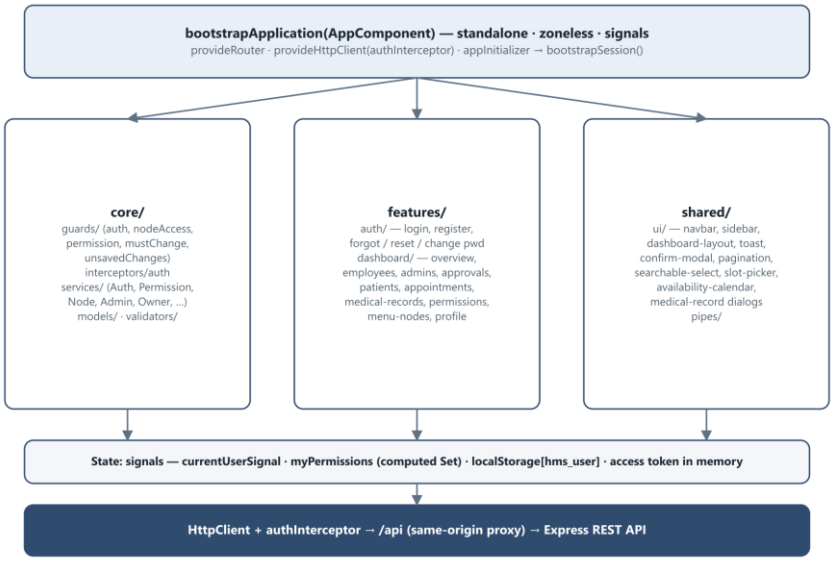

Figure 5.1: Angular Application Architecture

5.3 ROUTING AND ROUTE GUARDS

All routes are lazily loaded through loadComponent and protected by small functional guards. The authGuard blocks unauthenticated access and preserves the intended URL for post-login redirection; the mustChangePasswordGuard forces first-login users to set a new password; the nodeAccessGuard confirms the caller's designation may reach a given module; the permissionGuard confirms a specific action permission before allowing create or edit routes; and the unsavedChangesGuard prompts before discarding a dirty form. The account owner bypasses node gating so it can never be locked out of administration.

Table 5.2: Angular Route Guards

Guard	Enforces
authGuard	Authenticated session; redirects to /login with returnUrl
mustChangePasswordGuard	First-login password change before dashboard access
nodeAccessGuard	Designation may reach the requested module (owner bypass)
permissionGuard(codes)	Specific action permission for create / edit routes
unsavedChangesGuard	Confirmation before leaving a modified form

5.4 HTTP INTERCEPTOR AND SESSION STATE

A single functional interceptor attaches the in-memory access token to every outgoing request. When a protected call returns 401, the interceptor triggers a single shared refresh request—coordinated with an RxJS shareReplay so that a burst of concurrent 401s produces exactly one refresh—and then retries the original request with the new token. A 403 raises an access-denied toast and redirects to the dashboard overview, and a network failure raises a network-error toast. The only persistent client state is a single localStorage entry caching the current user (including the permission list); the access token lives only in memory and the refresh token is an HttpOnly cookie the JavaScript never sees.

5.5 SERVICES, SIGNALS, AND PERMISSION-BASED UI

Each backend domain has a dedicated service that exposes typed methods mapping directly to endpoints. Shared state is held in signals: the authenticated user is a signal, and the caller's permissions are exposed as a computed read-only set derived from it. Because components use OnPush change detection over these signals, any permission change—at login, on refresh, or via the sidebar's periodic re-sync—automatically re-renders permission-gated controls. Rather than a structural directive, the UI gates buttons and sections with computed booleans such as canCreate or canDelete evaluated against the permission set. Table 5.3 lists the principal services.

Table 5.3: Angular Services

Service	Backend Base	Responsibility
AuthService	/api/auth	Login, registration, password flows, session bootstrap, token refresh
PermissionService	/api/permissions	Effective permissions and the owner permission editor
NodeService	/api/nodes	Sidebar nodes and node management (cached)
AdminService	/api/admin	Employees, approvals, audit logs, profile-change requests
OwnerService	/api/owner	Administrator lifecycle
EmployeeService	/api/employees	Self profile, doctor list, profile-update requests
PatientService	/api/patients	Patient CRUD and search
AppointmentService	/api/appointments	Appointment lifecycle and slot queries
MedicalRecordService	/api/medical-records	Record lifecycle

Service	Backend Base	Responsibility
DashboardService	/api/dashboard	Role-aware statistics

5.6 FORMS, VALIDATION, AND ERROR HANDLING

Every data-entry page uses reactive forms backed by a shared library of custom validators covering names, phone numbers, password complexity, medical-registration numbers, and availability-slot ordering and conflict detection. Forms autosave drafts to an in-memory store that strips password fields. API errors surface through a central error-handler service that prefers field-level validation messages and falls back to the response message, displayed via the custom toast component; success paths raise confirmation toasts. Status colors follow a consistent convention in which only successful states are green and failures are red.

6. REACT NATIVE MOBILE APPLICATION ARCHITECTURE

The patient-facing client is a React Native application built with Expo SDK 56 and expo-router. Navigation is file-based with a custom, authentication-aware tab bar; server state is managed with TanStack Query; and small pieces of client state live in Zustand stores. The application communicates with the backend through a hand-written fetch wrapper rather than a third-party HTTP library.

6.1 TECHNOLOGY STACK

Table 6.1: Mobile Technology Stack

Technology	Purpose
React Native 0.85 / Expo SDK 56	Cross-platform runtime and toolchain
expo-router	File-based routing with a custom tab bar
TanStack Query	Server-state caching, invalidation, and refetch
Zustand	Lightweight client state (auth, nav guard, modal)
fetch (custom apiClient)	HTTP transport with single-flight token refresh
expo-secure-store	Encrypted storage of access and refresh tokens
@react-native-community/datetimepicker	Date and time selection

6.2 STRUCTURE, NAVIGATION, AND SCREENS

The root layout prepares fonts and session state behind a splash screen, wraps the tree in the query and keyboard providers, and registers global handlers so a dead session or forced password change redirects appropriately. The custom tab bar shows public tabs (Home, Login, Register) when logged out and the full application tabs (Home, Appointments, Records, Profile) when logged in, redirecting away from protected routes if the session ends. Figure 6.1 depicts the structure and Table 6.2 lists the screens.

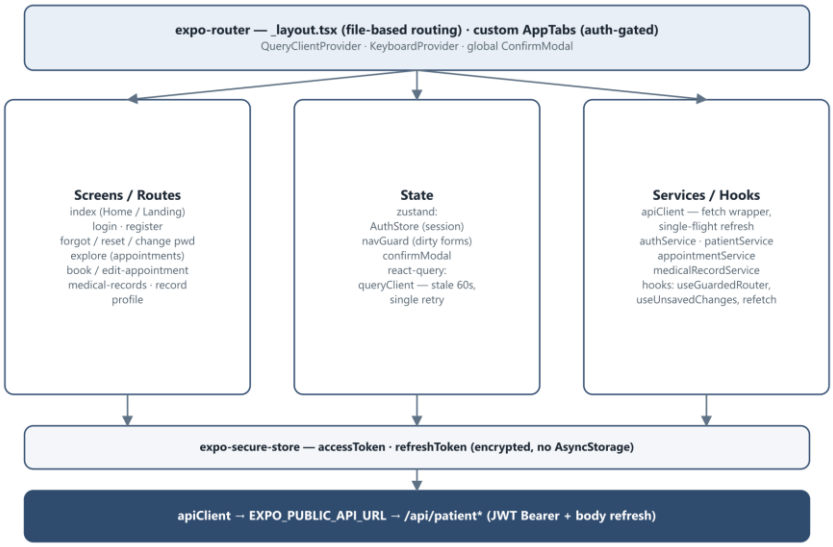

Figure 6.1: React Native Application Architecture

Table 6.2: Mobile Screens

Screen	Purpose
Landing / Home	Public landing when logged out; dashboard of upcoming visits when logged in
Login / Register	Patient authentication and self-registration
Explore (Appointments)	Book new visits and manage existing ones with status filters
Book / Edit Appointment	Shared form for booking and rescheduling
Medical Records / Record	Paginated finalized records and single-record detail
Profile	View and inline-edit contact details; change password; log out
Forgot / Reset / Change Password	Credential recovery and rotation

6.3 AUTHENTICATION, STORAGE, AND SHARED LOGIC

Both tokens are held in the encrypted secure store; no patient identifiers are persisted in plain storage. The apiClient prefixes the base URL, attaches the Bearer token, and unwraps the response envelope. On a 401 it performs a single-flight refresh, saves the rotated tokens, and replays the original request once; if refresh fails it clears the tokens and routes to login, and a 403 carrying a password-change code routes to the change-password screen. Cross-cutting concerns are centralized in hooks: a guarded router that blocks navigation away from dirty forms, an unsaved-changes hook that also intercepts the Android back button, and a focus hook that refetches stale queries when a screen regains focus.

6.4 STATE MANAGEMENT AND ERROR HANDLING

Server data is owned by TanStack Query with a sixty-second stale time and a single retry, while Zustand holds the login flag, the form-dirty flag, and the promise-based confirmation modal. The appointment cancellation flow uses optimistic updates with rollback on failure, and errors are surfaced through native alerts driven by a centralized message catalog.

7. API ARCHITECTURE

The API is a conventional REST interface. Endpoints are grouped by domain under a shared /api prefix, with a parallel /api/patient tree for the mobile client. Every response—success or error—uses the same envelope carrying a success flag, status code, message, and either a data payload or an error list, so both clients handle responses uniformly.

7.1 AUTHENTICATION AND SESSION ENDPOINTS

Table 7.1: Authentication Endpoints

Method & Path	Description
POST /api/auth/login	Authenticate staff; issue access token and refresh cookie
POST /api/auth/self-register	Register a new staff member (pending approval)
POST /api/auth/refresh	Rotate the refresh cookie and issue a new access token
POST /api/auth/logout	Revoke the refresh token and clear the cookie
PUT /api/auth/change-password	Change password (also used for forced first-login)
POST /api/auth/forgot-password · /reset-password	Begin and complete a password reset by email
GET /api/auth/me	Return the current staff profile and permissions

7.2 STAFF, OWNER, AND EMPLOYEE ENDPOINTS

Table 7.2: Administration Endpoints

Method & Path	Description
POST /api/admin/create-employee	Create a staff account with a temporary password
GET /api/admin/employees	List employees (aggregated, filtered, paginated)
PUT /api/admin/approve-employee · reject-employee/:code	Approve or reject a pending registration
GET /api/admin/audit-logs	List audit entries (permission-gated)
GET /api/admin/profile-change-requests	List pending self-service profile edits
PUT /api/admin/approve-profile-change/:id	Apply a requested profile change
POST /api/owner/create-admin	Owner creates an administrator account
PUT /api/employees/update-profile	Self profile update (direct or via request)

7.3 PATIENT AND APPOINTMENT ENDPOINTS

Table 7.3: Patient and Appointment Endpoints

Method & Path	Description
POST /api/patients/create-patient	Register a patient (staff-managed)
GET /api/patients / /search	List or search patients (paginated)
PUT/DELETE /api/patients/:UHID	Update or soft-delete a patient
POST /api/appointments/create-appointment	Book an appointment
GET /api/appointments / /my	List all or the doctor's own appointments
GET /api/appointments/booked-slots	Query taken slots for a doctor and date
PUT /api/appointments/:id/cancel	Cancel with a reason
PUT /api/appointments/:id/unattended	Mark a booked visit as unattended

7.4 MEDICAL RECORD, DASHBOARD, AND METADATA ENDPOINTS

Table 7.4: Records, Dashboard, and Metadata Endpoints

Method & Path	Description
POST /api/medical-records	Create a draft or finalized record
PUT /api/medical-records/:id	Update, verify, or finalize a record
GET /api/medical-records	List records (scoped by permission)
GET /api/dashboard/stats	Role-aware dashboard statistics
GET /api/nodes/my-nodes	Sidebar nodes for the caller's designation
GET /api/permissions/my-permissions	The caller's effective permission codes
PUT /api/permissions/update-permissions/:designation	Owner edits a designation's permissions

7.5 MOBILE (PATIENT) ENDPOINTS

Table 7.5: Patient-App Endpoints

Method & Path	Description
POST /api/patient/auth/login · /register	Patient authentication and self-registration

Method & Path	Description
POST /api/patient/auth/refresh · /logout	Body-based refresh and revocation
GET/PUT /api/patient/me	View and update the patient's own profile
GET /api/patient/doctors · /booked-slots	Doctor discovery and availability
GET/POST/PUT /api/patient/appointments	List, book, reschedule, and cancel own visits
GET /api/patient/medical-records	View own finalized records

8. AUTHENTICATION AND AUTHORIZATION

8.1 JWT AUTHENTICATION AND REFRESH TOKENS

Authentication uses JSON Web Tokens signed with HS256. Access tokens are short-lived (fifteen minutes) and carry the caller's identity, roles, and designation; staff and patients are signed with separate secrets so a token minted for one audience can never be replayed against the other, and the verifier pins the algorithm to prevent algorithm-confusion attacks. Refresh tokens are opaque random values, never JWTs: only their SHA-256 hash is stored, they are grouped into families, and each use rotates the token and invalidates its predecessor. Presenting an already-rotated token is treated as theft and revokes the entire family. A MongoDB TTL index purges expired tokens automatically. Figure 8.1 shows the flow.

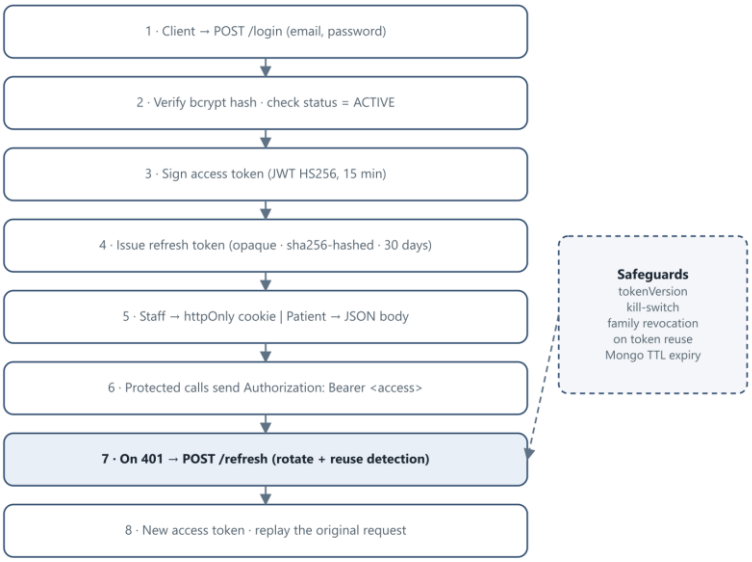

Figure 8.1: Authentication and Refresh Token Flow

Every authenticated request is re-validated against the database, not merely against the token signature. The middleware confirms the account still exists, is active, and carries the expected token version. A monotonic token-version counter on each account acts as a global kill switch: changing or resetting a password increments it, instantly invalidating all outstanding access tokens for that account.

8.2 SESSION MANAGEMENT ACROSS CLIENTS

The two clients manage sessions differently to suit their platforms. The Angular application keeps its refresh token in a first-party HttpOnly cookie scoped to the auth path, which the browser sends automatically and JavaScript cannot read; because the web app and API share an origin through a same-origin /api proxy, this cookie remains first-party. The mobile application has no cookie jar, so it stores both tokens in the device's encrypted secure store and sends the refresh token in the request body. Both approaches share the same rotation, reuse-detection, and expiry machinery on the server.

8.3 TWO-LAYER ROLE-BASED ACCESS CONTROL

Authorization is enforced in two independent layers on top of authentication. The first layer, the node door, checks whether the caller's designation is permitted to reach a functional module at all—mirroring which items appear in that user's sidebar. The second layer, the action permission, checks whether the caller holds the specific permission required by the operation, such as CREATE_APPOINTMENT or FINALIZE a medical record. Permissions are stored one document per designation and cached in memory for speed, with the cache invalidated whenever an administrator edits them. The account owner bypasses both layers by design. Controllers may additionally perform record-level checks—confirming, for instance, that a doctor only touches their own patients. Figure 8.2 illustrates the decision path.

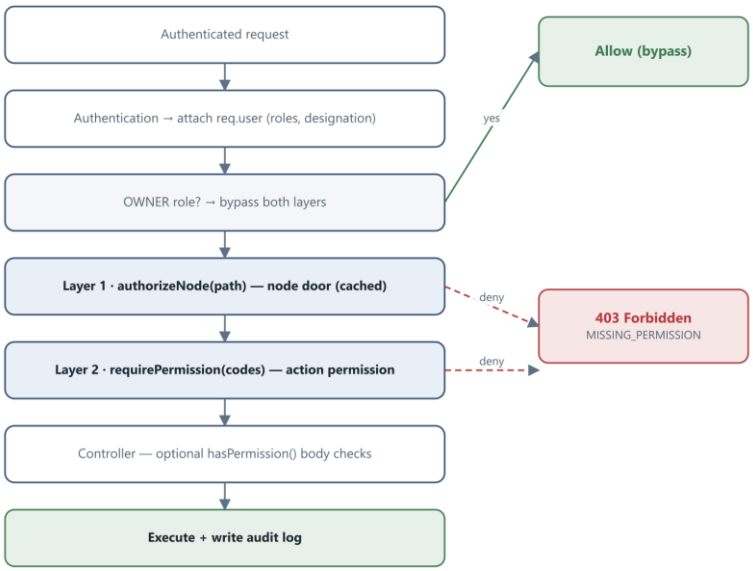

Figure 8.2: Two-Layer RBAC Authorization Flow

This design separates reachability from capability: removing a module from a designation automatically strips its action permissions, and because client and server evaluate the same permission codes, the interface a user sees always matches what the API will allow.

9. FUNCTIONAL MODULES

Each functional module spans a backend controller, web pages, and, where relevant, a mobile screen. Section 10 traces the workflows that connect them.

9.1 EMPLOYEE, ADMINISTRATOR, AND PATIENT MANAGEMENT

Administrators create staff accounts with temporary passwords, list and search employees through an aggregation pipeline joining the user and employee collections, and update or soft-delete records, while the account owner manages administrators through a dedicated module; doctor records carry consultation fees and weekly availability slots, and changing availability automatically cancels now-out-of-schedule appointments. Staff likewise register patients—generating a unique identifier and emailing temporary credentials—search across name, phone, email, and identifier, update contact details, and soft-delete records, and patients maintain their own contact information from the mobile app.

9.2 APPOINTMENT MANAGEMENT

Appointments move through booked, completed, canceled, and unattended states. Booking validates the slot against the doctor's availability and joining date, a six-month horizon, and existing bookings. Receptionists manage all appointments while doctors act only on their own; cancellation captures a reason, and completion follows the finalization of a medical record.

9.3 MEDICAL RECORD MANAGEMENT

Medical records capture the chief complaint, symptoms, diagnosis, advice, prescription items, and observations. A record may be a draft or finalized; finalizing requires the appropriate permission and the assigned doctor, and it completes the appointment. Staff may prepare a draft for a doctor to verify and finalize, supporting a two-person workflow. Records are soft-deleted, and patients see only finalized records.

9.4 APPROVALS, DASHBOARD, AND AUDITING

Two approval queues protect sensitive changes: self-registered staff remain pending until approved, and self-service profile edits by ordinary staff become change requests an administrator must approve, while privileged roles edit directly; both queues appear together on the approvals page with a difference viewer. The dashboard returns role-specific statistics, every privileged action is written to an immutable audit log that administrators can browse

(with failed logins and token reuse flagged as failures), listing endpoints share a common pagination helper, search endpoints escape input before building case-insensitive queries, and notifications are emailed. A billing model exists in the schema but is not yet wired to any route.

10. BUSINESS WORKFLOWS

10.1 LOGIN AND PERMISSION LOADING

A user submits credentials; the API verifies the password hash, confirms the account is active, records the login time, and returns an access token together with the user's profile and effective permissions, issuing the refresh token as a cookie for staff or in the body for patients. The client caches the user, and every subsequent navigation and control is gated by the loaded permissions.

10.2 REGISTRATION AND APPROVAL

Staff either self-register—creating a pending account that awaits administrator approval—or are created directly by an administrator, in which case they receive a temporary password by email and are confined to the change-password flow until they choose a new one. Approval activates the account and notifies the applicant; rejection soft-deletes it. Patients self-register from the mobile app and are active immediately.

10.3 APPOINTMENT AND RECORD LIFECYCLES

An appointment is booked against a validated slot, may be rescheduled or canceled, and is marked unattended if the patient does not appear. When the treating doctor finalizes the associated medical record, the appointment is automatically completed. A record may pass through a draft stage prepared by staff and then be verified and finalized by the assigned doctor, at which point the patient is notified and can view it.

10.4 PROFILE UPDATES AND PERMISSION CHECKS

When an ordinary staff member edits their profile, the change is captured as a request for administrator approval; a privileged user's edit applies immediately. Throughout the system, every mutating request is checked first for module reachability and then for the specific action permission, so the same policy governs both what a user can see and what they can do.

11. SYSTEM INTEGRATION

The three applications integrate through the shared REST API, which is the sole point of contact with the database and external services. This section describes each integration boundary.

11.1 WEB AND MOBILE CLIENTS TO THE API

The Angular client integrates over same-origin HTTPS: a build-time proxy in development and a platform rewrite in production both map /api to the backend, keeping the refresh cookie first-party and eliminating cross-site cookie concerns. The mobile client integrates over HTTPS to a configured base URL, authenticating with a Bearer access token and refreshing through a body-based token. Both rely on the identical server-side authorization pipeline, so the two clients can never diverge in what they are permitted to do.

11.2 API TO DATABASE AND EXTERNAL SERVICES

The API connects to MongoDB through a single pooled Mongoose connection that is reused across requests and, in the serverless deployment, across warm invocations. Transactional email is sent through the Brevo HTTP API. Because MongoDB is the single source of truth, an action taken on one client is immediately consistent for the others—an appointment a patient books on the phone appears instantly to a receptionist on the web.

12. DEPLOYMENT ARCHITECTURE

HMS supports two complementary deployment paths from the same repository: a managed serverless deployment on Vercel and a self-hosted deployment on an AWS EC2 virtual machine driven by GitHub Actions. Figure 12.1 shows both.

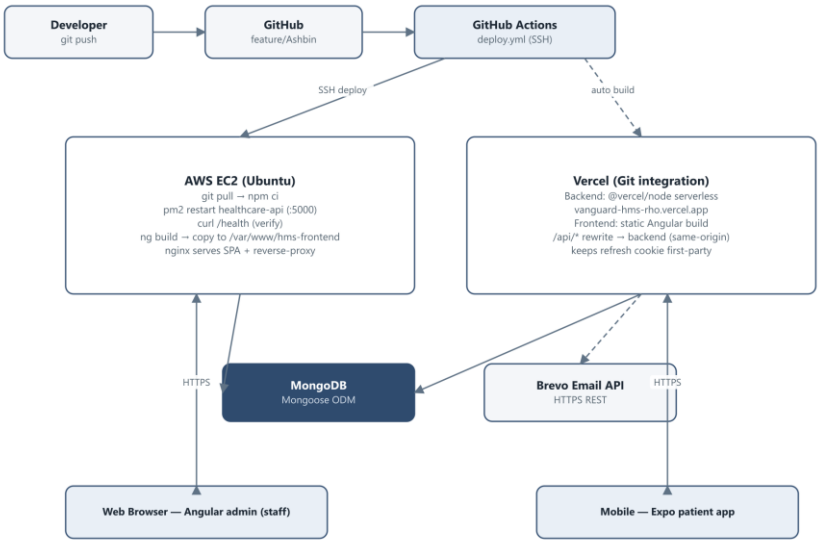

Figure 12.1: Deployment Architecture

12.1 VERCEL SERVERLESS DEPLOYMENT

For the managed path, the backend is packaged as a serverless function that wraps the same Express application and reuses the pooled database connection, while the Angular build is served as a static site. The web project rewrites /api requests to the backend deployment, giving the browser a same-origin API and keeping the refresh cookie first-party. This path is provisioned through Vercel's Git integration rather than a workflow file.

12.2 AWS EC2 DEPLOYMENT AND CI/CD

For the self-hosted path, a GitHub Actions workflow triggers on pushes to the working branch and connects to the EC2 host over SSH. It resets the checkout, pulls the latest code, installs backend dependencies with a clean install, and restarts the API under the pm2 process manager with an environment reload. It then performs a process-status check and an HTTP health check against the local /health endpoint, builds the Angular application, verifies the build output, copies it into the web root served by the reverse proxy, and finally verifies that the deployed

index file is present. Each of these guard steps fails the deployment early if a stage does not succeed, making the pipeline safe to run automatically.

12.3 ENVIRONMENT CONFIGURATION

The backend validates its environment at startup and refuses to boot if a required variable is missing or if either signing secret is too short. Table 12.1 lists the principal configuration values; the mobile application is configured separately through a single public base-URL variable.

Table 12.1: Principal Environment Variables

Variable	Role
MONGO_URI	MongoDB connection string
JWT_SECRET / JWT_PATIENT_SECRET	Separate HS256 signing secrets for staff and patients
JWT_EXPIRES_IN	Access-token lifetime (15 minutes)
REFRESH_TOKEN_EXPIRES_DAYS	Refresh-token lifetime (30 days)
FRONTEND_URL	Allowed CORS origin for the web client
BREVO_API_KEY / EMAIL_USER	Transactional email credentials and sender
PORT	API listening port (default 5000)

13. SECURITY

Security is enforced consistently at every layer, from transport and storage through authentication, authorization, and input handling.

13.1 CREDENTIALS AND TOKENS

Passwords are never stored in plain text; they are hashed with bcryptjs using ten salt rounds for staff and twelve for patients. Access tokens are HS256 JWTs with a fifteen-minute lifetime, signed with distinct secrets per audience and verified with a pinned algorithm and a per-request database re-check of account status and token version. Refresh tokens are opaque, stored only as SHA-256 hashes, rotated on every use with family-wide revocation on reuse, and expired automatically by a TTL index. Password-reset tokens and codes are likewise hashed and expire after fifteen minutes, and reset responses are deliberately neutral to avoid revealing whether an email exists.

13.2 AUTHORIZATION AND INPUT HANDLING

The two-layer role-based access model described in Section 8 governs every privileged endpoint, backed by controller-level record scoping. All mutating requests pass through express-validator rule sets and Mongoose schema constraints, and any user input used in a search is escaped before being placed into a query, preventing injection through regular expressions. The centralized error handler ensures failures never leak internal detail to the client.

13.3 TRANSPORT, HEADERS, AND STORAGE

The API sets hardened HTTP headers through Helmet, restricts cross-origin access to the configured frontend origin while permitting credentials for the refresh cookie, and throttles authentication and password-reset routes to resist brute-force attempts. The staff refresh cookie is HttpOnly and path-scoped and is marked secure in production; on mobile, tokens are held in the platform's encrypted secure store rather than ordinary storage. Together these measures protect credentials in transit, at rest, and in use.

14. ERROR HANDLING

Error handling is centralized in each tier so that failures are reported consistently and never leave the system in an ambiguous state.

14.1 BACKEND, WEB, AND MOBILE HANDLING

On the backend, controllers throw a typed AppError, and Express 5 forwards any rejected promise to a single error middleware that maps AppError instances, Mongoose validation and cast errors, duplicate-key violations, and malformed JSON to the correct status code and renders the standard envelope. The Angular interceptor handles cross-cutting cases—silent token refresh on 401, an access-denied redirect on 403, and a network toast on connectivity loss—while component-level errors flow through a shared handler that prefers field messages. The mobile client raises native alerts from a centralized message catalog and uses optimistic updates with rollback for cancellations.

14.2 VALIDATION AND COMMON RESPONSES

Validation runs before business logic on every mutating route, returning a 422 with field-level messages that the clients render inline. Table 14.1 lists the common error responses and how the clients react.

Table 14.1: Common Error Responses

Scenario	Status	Client Handling
Invalid credentials	401	Toast or alert: invalid email or password
Missing or expired token	401	Silent refresh, then redirect to login on failure
Insufficient permission	403	Redirect to overview / access-denied
Invalid request body	422	Field-level validation messages
Duplicate key	409	Conflict message
Resource not found	404	Not-found message
Server error	500	Generic error message

15. PERFORMANCE CONSIDERATIONS

15.1 DATA AND API EFFICIENCY

Listing endpoints are paginated through a shared helper that bounds page size, and employee listings use a single aggregation pipeline with a faceted stage to return both rows and totals in one round trip. Unique and TTL indexes support fast lookups and automatic cleanup of expired sessions. Frequently read authorization data—permissions and node access—is served from in-memory caches that are invalidated only when an administrator changes them, avoiding a database read on every request while remaining immediately consistent after edits.

15.2 CLIENT EFFICIENCY AND MAINTAINABILITY

The Angular application is zoneless and uses OnPush change detection over signals, so the framework re-renders only what actually changed, and its routes are lazily loaded to keep the initial bundle small. Token refresh is coordinated so that concurrent failures trigger a single network call. The mobile application caches server state with TanStack Query using a stale-time window that avoids redundant fetches while keeping data fresh on focus. Across all tiers, clear layering, centralized cross-cutting concerns, and a single shared response contract keep the codebase maintainable and make the behaviour of the system easy to reason about and extend.

16. FUNCTIONAL MODULE SUMMARY

Table 16.1 maps each major module to its implementation across the three applications, showing how a single backend capability is surfaced to staff and patients through their respective clients.

Table 16.1: Functional Module Summary

Module	Backend	Angular Web	Mobile
Authentication	authController	auth pages	login / register
Employee & Admin	admin / owner controllers	employees, admins	—
Patient	patient controller	patients pages	profile
Appointment	appointment controller	appointments pages	explore, book, edit
Medical Record	medicalRecord controller	medical-records	records, record
Approvals	admin controller	approvals	—
Profile Change	employee controller	profile, approvals	profile
Dashboard	dashboard controller	overview	home
Audit Log	admin controller	overview feed	—
RBAC & Nodes	permission / node controllers	permissions, menu-nodes	—

17. CONCLUSION

The Hospital Management System is a cohesive three-tier platform whose defining characteristic is a single authoritative backend that encodes the hospital's data model and access-control policy once and enforces it uniformly for every client. The Express API provides a layered, well-validated REST interface secured by short-lived JWTs, rotating refresh tokens with reuse detection, and a two-layer role-based access model that separates module reachability from action capability. The Angular administrative application delivers a fast, signal-driven, permission-aware interface for staff, and the Expo mobile application extends self-service booking and record access to patients, each integrating through the same authenticated API and the same authorization pipeline.

Supported by soft deletion, a complete audit trail, atomic identifier generation, and a dual serverless-and-self-hosted deployment strategy with automated health-checked delivery, the architecture is secure, consistent, and maintainable. Its clear separation of concerns and shared API contract make the system straightforward to operate today and to extend as the hospital's needs grow.